

Ce Que Je Fais — Journal des Corrections

Phase 1 : Fiabilisation multi-tenant & compatibilité PostgreSQL

C1 — Correction AuditController (lecture depuis `projects`)

Date : 2026-06-22

Section : `app/Http/Controllers/Gel/AuditController.php`, vues GEL/Company audit

Problème résolu : L'AuditController lisait les données d'audit depuis un mauvais modèle ou sans isolation multi-tenant, ce qui pouvait exposer les activités d'un client à un autre. Les vues d'audit GEL et Company n'étaient pas correctement isolées par `client_id`.

Impact :  Sécurité multi-tenant renforcée — chaque client ne voit que ses propres entrées d'audit. Les cabinets (GEL) voient les projets qui leur appartiennent.

Fichiers modifiés : AuditController, vues d'audit GEL et Company

C2 — Contrainte UNIQUE sur `services.client_subdomain + company_id`

Date : 2026-06-22

Section : `migrations / schema services`

Problème résolu : Absence de contrainte d'unicité sur le couple (`client_subdomain, company_id`) dans la table `services`. Cela permettait des doublons de sous-domaine pour une même entreprise, causant des conflits de routage et d'identification du tenant.

Impact :  Intégrité des données garantie au niveau base. Empêche les doublons de sous-domaines par entreprise. Les tentatives d'insertion en double lèvent une exception explicite.

C3 — Liste comptable entreprise utilise les codes SYSCOHADA comme discriminateur de type

Date : 2026-06-22

Section : `CompanyAccountingController, AccountingController` (Company), liste des comptes

Problème résolu : Les listes de comptes dans l'interface Company (entreprise) utilisaient un système de types français arbitraires (Actif/Passif/Produits/Charges) alors que le plan comptable SYSCOHADA s'identifie par classes (1, 2, 3... 8). Les deux systèmes coexistaient mais le discriminateur de type pour les listes entreprise n'était pas synchronisé avec les codes de classe SYSCOHADA, causant des affichages vides ou incorrects.

Impact :  Les listes de comptes entreprise utilisent désormais le numéro de classe SYSCOHADA (1-8) comme discriminateur, en phase avec le plan comptable OHADA. Cohérence totale entre GEL et Company.

Correction connexe — Période TVA : Les déclarations TVA utilisaient par erreur la période de l'exercice fiscal entier (`date_start` → `date_end`) au lieu du mois concerné. Corrigé dans `TaxCalculationService.php`, `CompanyAccountingController::computeTva()` et `TaxDeclarationController::calculerTva()` pour utiliser `startOfMonth()` → `lastOfMonth()`.

Correction connexe — Champ DAE : `DaeAgendaEvent` utilise `start_at` et non `start`. Corrigé dans `DaeDashboardController::stats()` et `CompanyDaeController::stats()`.

C4 — Protection RLS `try/finally` dans tous les seeders

Date : 2026-06-22

Section : `database/seeders/` — 6 seeders corrigés

Problème résolu : PostgreSQL Row-Level Security (RLS) utilise une variable de session `app.client_id` pour isoler les données entre clients. Les seeders définissaient cette variable en début d'opération mais ne la réinitialisaient pas en cas d'exception. Une exception en cours de seeding laissait le contexte RLS actif, corrompant les opérations suivantes (écriture dans le mauvais client).

Impact :  Robustesse des seeders garantie. Même en cas d'exception, `app.client_id` est toujours remis à `'0'`. Élimine les corruptions silencieuses de données entre clients lors des opérations de seeding.

Détail par seeder :

Seeder	Statut	Correctif
<code>SyscohadaChartSeeder</code>	 Réparé	Ajout <code>try/finally</code> dans <code>createForClient()</code>
<code>EdenStoreFolderSeeder</code>	 Réparé	Ajout <code>try/finally</code> dans <code>createStructureForClient()</code> , suppression nested <code>try</code>
<code>DemoCompanySeeder</code>	 Réparé	Correction pile d'accolades PHP (LIFO) — <code>} finally {</code> fermait <code>foreach</code> au lieu de <code>try</code>
<code>AccountingDemoSeeder</code>	 Réparé	Même problème LIFO — découplage des blocs <code>} et }</code> <code>finally {</code>
<code>DaeDemoSeeder</code>	 Réparé	Ajout accolade de classe manquante après <code>finally</code>
<code>CrescendoDemoSeeder</code>	 Réparé	Ajout accolade de classe manquante après <code>finally</code>

Problème connexe détecté : PHP ferme les accolades en LIFO (Last In, First Out), pas par niveau d'indentation. Plusieurs seeders avaient `} finally {` (8 espaces) qui fermait le bloc de contenu au même niveau au lieu du `try` (8 espaces aussi). La solution est de séparer : `} \n } finally {` où le `}` à 8sp ferme le contenu et `} finally {` à 4sp ferme le `try`.

C5 — Centralisation `getClientId()` dans `BaseCompanyController` + méthode `authorizeClient()`

Date : 2026-06-22

Section : app/Http/Controllers/Company/ — 17 controllers refactorisés

Problème résolu : Chacun des 17 Company controllers implémentait sa propre méthode privée `getClientId()` pour récupérer l'ID du client actif — 17 copies quasi-identiques (violation DRY). Pire, elles utilisaient toutes `$user->client_id` (entreprise par défaut) au lieu de `$user->active_client_id ?? $user->client_id` (contexte multi-entreprise), ce qui ne respectait pas la logique du middleware `EnsureCompanyAccess`. Une méthode `authorizeClient()` manquait pour bloquer explicitement les accès跨-client.

Solution :

- 1. Création de `BaseCompanyController` (classe abstraite étendant `Controller`) avec :
 - `getClientId(): int` — version centralisée qui utilise `active_client_id ?? client_id` (même logique que `EnsureCompanyAccess`)
 - `authorizeClient(?int $targetClientId)` — vérification explicite de propriété client
- 2. Refactorisation des 17 controllers Company pour étendre `BaseCompanyController`
- 3. Suppression des 17 méthodes `getClientId()` dupliquées

Impact :  DRY — 17 copies → 1 centralisée.  Logique `active_client_id` maintenant cohérente avec le middleware.  Protection跨-client via `authorizeClient()`.  Changement transparent — signature `getClientId()` inchangée pour les appelants.

Détail par controller :

Controller	Pattern supprimé	Méthode de base utilisée
FiscalYearController	Private, retourne <code>\$user->client_id</code>	 <code>\$this->getClientId()</code>
FixedAssetController	Private, retourne <code>\$user->client_id</code>	 <code>\$this->getClientId()</code>
TvaController	Private, retourne <code>\$user->client_id</code>	 <code>\$this->getClientId()</code>
BankReconciliationController	Private, retourne <code>\$user->client_id</code>	 <code>\$this->getClientId()</code>
AccountingController	Private, retourne <code>\$user->client_id</code>	 <code>\$this->getClientId()</code>
AiController	Private avec <code>(int)</code> cast	 <code>\$this->getClientId()</code>
CaisseController	Private avec <code>(int)</code> cast	 <code>\$this->getClientId()</code>
CrmController	Private, retourne <code>\$user->client_id</code>	 <code>\$this->getClientId()</code>

Controller	Pattern supprimé	Méthode de base utilisée
GedController	Private avec (int) cast	✓ \$this->getClientId()
HumanResourcesController	Private, retourne \$user->client_id	✓ \$this->getClientId()
InvoiceController	Private, retourne \$user->client_id	✓ \$this->getClientId()
LegalController	Private, retourne \$user->client_id	✓ \$this->getClientId()
ProjectController	Private, retourne \$user->client_id	✓ \$this->getClientId()
UserController	Private avec (int) cast	✓ \$this->getClientId()
CompanyAccountingController	Private one-liner avec (int)	✓ \$this->getClientId()
CompanyDaeController	Private nullable : ? int (retourne null)	✓ \$this->getClientId() (désormais abort 403 si pas de client, plus robuste)
CompanyRhController	Protected nullable (retourne null)	✓ \$this->getClientId() (désormais abort 403 si pas de client, plus robuste)

Fichier créé : app/Http/Controllers/Company/BaseCompanyController.php

C6 — Centralisation getClientId() — Module RH (BaseRhController)

Date : 2026-06-22

Section : app/Http/Controllers/Modules/Rh/ — 9 controllers refactorisés

Problème résolu : Les 8 controllers RH implémentaient chacun une méthode getClientId() quasi-identique (violation DRY). RhDashboardController avait le pattern inline \$clientId = \$request->input('client_id') ?: Auth::user()?->client_id. Aucun abort(403) en cas de client_id manquant — risque de requêtes sans scope tenant.

Solution :

1. Création de BaseRhController avec getClientId(Request): int centralisé (abort 403 si pas de client)
2. Refactorisation des 9 controllers RH pour étendre BaseRhController
3. Suppression des 8 méthodes getClientId() dupliquées

Impact : ✓ DRY — 8 copies → 1 centralisée. ✓ `abort(403)` ajouté pour sécurité multi-tenant. ✓
Changement transparent.

Fichier créé : `app/Http/Controllers/Modules/Rh/BaseRhController.php`

Fichiers modifiés (9) :

- `RhEmployeesController`, `RhLeavesController`, `RhPayrollsController`,
`RhAttendanceController`
- `RhTrainingsController`, `RhContractsController`, `RhExpensesController`,
`RhAlertsController`
- `RhDashboardController`

C7 — Centralisation `getClientId()` — Module DAE (BaseDaeController)

Date : 2026-06-22

Section : `app/Http/Controllers/Modules/Dae/` — 10 controllers refactorisés (+1 exclu)

Problème résolu : Les controllers DAE utilisaient un pattern inline `if ($request->filled('client_id')) $query->where('client_id', $request->client_id)` pour filtrer par client, ce qui rendait le scope multi-tenant optionnel — un appel sans `client_id` exposait les données de tous les clients. Les validations `'client_id' => 'required|exists:clients,id'` dans les store/update étaient redondantes et contournaient la logique centralisée.

Solution :

1. Création de `BaseDaeController` avec `getClientId(Request): int` centralisé
2. Remplacement des `if ($request->filled('client_id'))` par `$query->where('client_id', $this->getClientId($request))` — scope obligatoire
3. Remplacement des `'client_id' => 'required|exists:clients,id'` par `$validated['client_id'] = $this->getClientId($request)` dans les store
4. `DaeDashboardController` exclu — pattern spécial GEL multi-client (`whereIn`)

Cas particuliers :

- `DaeModelesController` — `client_id` nullable conservé (modèles partagés entre clients)
- `DaeDocumentsController` — `upload()` garde `$clientId` pour usage futur
- `DaeEmailsController` — `repondre()` et `stats()` adaptés au nouveau pattern

Impact : ✓ Scope multi-tenant désormais obligatoire. ✓ DRY — pattern inline remplacé par méthode centralisée. ✓ Pas de régression sur les cas particuliers.

Fichier créé : `app/Http/Controllers/Modules/Dae/BaseDaeController.php`

Fichiers modifiés (10) :

- `DaeAgendaController`, `DaeConformiteController`, `DaeContratsController`,
`DaeCourriersController`
- `DaeDocumentDossiersController`, `DaeDocumentsController`, `DaeEmailsController`,
`DaeModelesController`

- `DaePersonnelController`, `DaeRapportsController`, `DaeTachesController`

Exclu : `DaeDashboardController` (pattern GEL multi-client)

C8 — Centralisation `getClientId()` — Module Legal (`BaseLegalController`)

Date : 2026-06-22

Section : `app/Http/Controllers/Modules/Legal/` — 9 controllers refactorisés

Problème résolu : Les controllers Legal utilisaient le pattern inline `$request->get('client_id', auth()->user()->client_id ?? 0)` avec fallback à `0` — aucune distinction entre l'absence de client et l'ID `0` qui n'existe pas, rendant le débogage difficile.

Solution :

1. Création de `BaseLegalController` avec `getClientId(Request): int` centralisé
2. Remplacement des appels inline par `$this->getClientId($request)` — `abort(403)` si client manquant
3. Suppression des `'client_id' => 'required|integer'` des validations store

Cas particulier :

- `LegalActsLibraryController` — `client_id` nullable conservé dans `store()` (modèles globaux partagés)

Impact : ☒ DRY — pattern inline remplacé. ☒ Comportement explicite : 403 si pas de client, plus de fallback silencieux à `0`.

Fichier créé : `app/Http/Controllers/Modules/Legal/BaseLegalController.php`

Fichiers modifiés (9) :

- `LegalActsLibraryController`, `LegalAssembliesController`, `LegalCompanyInfoController`
 - `LegalComplianceController`, `LegalContratsController`, `LegalDashboardController`
 - `LegalDossiersController`, `LegalLitigationsController`, `LegalRegistresController`
-

C9 — Harmonisation GEL Accounting (`BaseGelAccountingController`)

Date : 2026-06-22

Section : `app/Http/Controllers/Gel/Accounting/` — 7 controllers refactorisés

Problème résolu : Les 7 controllers GEL Accounting utilisaient chacun leur propre pattern pour récupérer le `client_id` — certains via validation `'client_id' => 'required|exists:clients,id'` dans le body (store), d'autres via route param `{clientId}` (GET), et d'autres n'avaient aucun scope client (post/destroy des journaux, clôture). Violation DRY et risque multi-tenant.

Solution :

1. Création de `BaseGelAccountingController` avec `getClientId(Request): int` centralisé :
 - Priorité : route param `{clientId}` → body `client_id`
 - `abort(403)` si aucun client trouvé
2. Refactorisation des 7 controllers pour étendre `BaseGelAccountingController`
3. Suppression des patterns dupliqués

Détail par controller :

Controller	Changement
<code>BaseGelAccountingController</code> (créé)	<code>getClientId()</code> centralisé — route <code>{clientId}</code> puis body <code>client_id</code> , abort 403
<code>AccountController</code>	<code>extends BaseGelAccountingController</code> , remplacement <code>client_id</code> validation par <code>\$this->getClientId()</code>
<code>JournalController</code>	<code>extends BaseGelAccountingController</code> , remplacement <code>client_id</code> validation + protection <code>post()/destroy()</code> avec scope client optionnel
<code>BudgetController</code>	<code>extends BaseGelAccountingController</code> , remplacement <code>client_id</code> validation dans <code>store()</code>
<code>ClosingController</code>	<code>extends BaseGelAccountingController</code> , ajout scope client optionnel sur <code>cloturer()/rouvrir()/inventaire()</code>
<code>TaxDeclarationController</code>	<code>extends BaseGelAccountingController</code> , remplacement <code>client_id</code> validation dans 5 méthodes <code>calculer*()</code>
<code>ReportController</code>	<code>extends BaseGelAccountingController</code> (inchangé — déjà via route param)
<code>PdfExportController</code>	<code>extends BaseGelAccountingController</code> (inchangé — déjà via route param)

Impact : ☒ DRY — 6 patterns dupliqués → 1 méthode centralisée. ☒ Scope multi-tenant renforcé sur les endpoints store/post/destroy qui en étaient dépourvus. ☒ Changement rétrocompatible (les méthodes GET avec `{clientId}` route param continuent de fonctionner).